

大模型分词技术以及有关的有趣现象

算法小小怪下士

2026 年 5 月 18 日

1 为什么需要分词？

自然语言以 text 的形式存在，但神经网络模型只能处理数值。因此，任何文本在进入模型之前，都必须先被映射为整数索引，将“符号”转化为“数字”。最简单的映射方式是按字符切分，即把每个字或标点视为一个独立的 token。这种方式虽然简单，但会导致序列极长，计算成本大幅上升，而且单个字符难以承载丰富的语义。另一种方式是按单词切分，将完整的词语映射为一个 token。但这会让词表变得极其庞大，并产生 OOV 问题，一旦遇到训练中未见过的词，模型便完全无法处理。

为了在计算效率、语义表达和泛化能力之间取得平衡，子词（Subword）分词成为主流方案。其核心思想是：将高频常用词保留为完整的 token，将低频生僻词拆分为更小的、可复用的子词单元。这样既控制了词表大小和序列长度，又能通过子词的组合来表示任意新词，显著缓解 OOV 问题。如今几乎所有现代大模型（如 GPT、LLaMA、BERT 等）都采用子词分词算法。

2 主流分词算法

在真正对文本进行子词拆分之前，现代分词器通常会先完成统一化，将原始文本转换为 Unicode 码点序列。例如，“你好”会被表示成 [20320, 22909]。这一步打破了中文、英文、表情符号等不同书写系统之间的壁垒，让分词器在底层看到的只是纯粹的数字，不再需要理解汉字的形态或字母的拼写，实现了语言无关的底层表示。像 SentencePiece 这样的分词工具，就把“所有输入先转换为 Unicode 字符”作为设计的核心前提。

2.1 BPE (Byte Pair Encoding)

BPE 的核心思想是：从字符级开始，迭代地合并训练语料中出现频率最高的相邻 token 对，逐步构建出更大的子词单元。这是一个贪心过程，每次合并只考虑当前的频率最高者，因此不能回溯，只能保证局部最优，而非全局最优。其基本步骤为：(1) 将每个字符视为一个 token，统计所有相邻 token 对的出现频率；(2) 合并频率最高的 token 对，生成一个新的 token，并更新整个语料中的 token 序列；(3) 重复上述合并过程，直到词表达达到预设大小。BPE 简单高效，被 GPT 系列、RoBERTa 等模型广泛采用。举例来说，“lower”和“lowest”这两个词，经 BPE 训练后可能拆分出“low”“er”“est”等可复用的子词单元。

2.2 WordPiece

WordPiece 的整体流程与 BPE 类似，都是从较小的词表出发，逐步扩充，但它的合并依据不是单纯的频率，而是最大化训练数据的似然。具体而言，每次选择加入词表的子词，是能使语言模型在训练数据上的概率提升最大的那一个。该方法通常使用 ## 前缀来标记非开头的子词，例如“playing”会被切分为“play”和“##ing”。BERT、DistilBERT 等模型就采用了 WordPiece 分词。

2.3 Unigram Language Model

与 BPE 和 WordPiece “从小词表逐步增大”的路线相反，Unigram 模型从一个极大的候选词表开始，然后逐步剪枝，去掉对整体损失函数贡献最小的子词，直至词表缩减到目标大小。其基本过程为：(1) 初始化一个尽可能完备的词表，包含所有可能的子词；(2) 利用语言模型和 EM 算法，评估每个子词在训练数据中的重要性；(3) 移除重要性最低的一定比例子词，重复以上步骤，直到达到目标词表大小。Unigram 的显著特点是支持概率化切分：同一段文本可以存在多种合理的分词路径，算法根据概率选择最优路径。这种灵活性使其能够更好地处理分词歧义。T5、XLM 等模型采用了该算法。

2.4 SentencePiece

传统 BPE 分词器（如 GPT-2 所用）通常依赖一个预分词步骤，即先用空格将文本切分成“单词”，再在每个单词内部进行 BPE 合并。对于以空格为天然词边界的英语来说，这很自然；但对于没有空格的中文、日文等语言，预分词无法产生有意义的“词”，只能把每个字当作一个独立单元，导致后续的 BPE 难以学到跨字的有效子词组合，词表效率和语义表达都大打折扣。SentencePiece 的核心创新在于彻底取消预分词步骤，直接面向原始文本（Unicode 码点序列）进行学习，不再依赖空格来提供词边界信息。同时，它将空格本身也视为一个普通字符并显式编码（通常用“`▯`”表示），使得分词结果可以无损可逆地还原回原始文本。这一设计让 SentencePiece 天然适用于中文、日文等无空格语言，也在多语言混合场景下表现得更为稳健。正因如此，SentencePiece 已成为 LLaMA、Mistral 等主流开源大模型的标准分词方案，标志着子词分词工具从“空格优先”正式迈向了“语言无关”的阶段。

3 分词对大模型的影响

3.1 不同模型的 tokenizer 大小

压缩率 = 原始文本的字符数 / 分词后得到的 Token 数量（或者反过来：同等文本下，Token 数量越少，压缩率越高）。高压压缩率意味着一个 Token 能“代表”更多字符，序列长度更短，显存占用减少、推理更快。BPE 配合大词表能提高压缩率，因为更大的词表意味着更细粒度的常用词被整体保留，文本切分更“粗糙”，从而用更少的 Token 表示相同的文本 → 压缩率提升。直观理解：大词表 ≈ 词典更厚，查一个字就能搞定的事不用拆成偏旁部首。

但是词表大了，这些参数全部需要训练和存储，会显著增加模型的总参数量、内存占用，以及训练时的计算和通信开销。因此，设计分词器时，需要在压缩效率和模型容量之间找一个平衡点。这就是你看到不同大模型 Tokenizer 压缩率差异很大的根本原因——它们选择了不同大小的词表，从而在效率与参数量之间做出了不同的取舍。下表对比了主流大模型的 tokenizer 大小：

模型	词表大小	备注
Gemma	256k	
GPT-4o / o1	200k	相比前代在非英语语言（如中文）上的压缩率大幅提升
Qwen 2.5	~152k	针对多语言优化，覆盖中英文等场景
Qwen 3.5	248k	
DeepSeek-V3	~129k (129,280)	词表比 V2 版本更大，以提升多语言效率
Llama 3	128k (128,256)	相比 Llama 2 的 32k 词表大幅提升
GLM3	65,024	相比早期 150k 词表缩小，优化加载速度
GLM4	151,328	
GLM5.1	154,880	

3.2 实际示例 (GPT-4 风格的分词)

输入: "Hello, how are you?" 分词结果: ["Hello", ",", " ", "how", " ", "are", " ", "you", " ", "?"] (注: 空格常合并到后续词的开头, 标点独立成 Token)。输入: "unhappiness" 分词结果: ["un", "happiness"] 或 ["un", "happi", "ness"], 取决于训练数据。

3.3 为什么现在大模型在迭代的时候几乎不迭代词表?

原因有二: 第一, 生态惯性。词表是模型家族的“基因”, 家族内成员共享同一套架构和词表。升级时保持词表不变, 能确保下游生态的稳定性, 工具 (如 LoRA 微调、量化方案) 和推理引擎可无缝兼容, 大大降低了升级成本。第二, 收益有限。在词表足够大 (如超过 100k) 时, 进一步扩充带来的压缩率提升已微乎其微。而且, 改变分词器会破坏已有语料的切分方式, 大量已处理数据必须重新 Tokenize。

3.4 分词技术与大模型的多语言能力

Token Tax 这一概念已被多项研究实证: 不同语言在同模型下被切分为显著不同数量的 Token, 形态复杂或非拉丁文字的语言承担更重的计算代价。Token 数量翻倍则训练成本和耗时增至四倍, 这被称为低资源语言面临的“Token 税”。一项覆盖 200 多种语言的大规模横语言评估发现, 拉丁文字语言的 Tokenization 效率普遍较高, 而非拉丁文字和形态复杂的语言往往承受 3-5 倍的相对 Token 化成本。并且基于 10 个大模型在 16 种非洲语言上的评估 (AfriMMLU) 发现, Token Fertility (每个词的平均 Token 数) 可以可靠预测模型准确率——Fertility 越高, 准确率越低。

4 Tokenizer 也有 Scaling Law 吗?

4.1 Scaling Laws with Vocabulary: Larger Models Deserve Larger Vocabularies 【港大】

词汇量对模型性能有显著影响, 存在一个与 FLOPs 相关的最优词汇量。当前大多数 LLM (如 Llama2-70B) 使用的词汇量偏小, 未能达到计算最优。因此需要回答: 给定计算预算, 最优词汇量是多少? 主要方法包括: (1) IsoFLOPs 分析与幂律拟合: 训练非词汇参数从 33M 到 1.13B 的模型, 固定 FLOPs 预算, 改变词汇量 (4K 96K), 拟合幂律关系, 得出词汇参数应比非词汇参数增长更慢 (指数 $\gamma \approx 0.84$)。 (2) 基于导数的快速估计。 (3) 损失函数的参数化拟合: 修改 Chinchilla 缩放定律, 引入词汇参数 N_v 和训练字符数 H , 设计损失函数:

$$\mathcal{L}_u = -E + \frac{A_1}{N_{nv}^{\alpha_1}} + \frac{A_2}{N_v^{\alpha_2}} + \frac{B}{[Hf(V)]^\beta}$$

通过实验数据拟合参数, 进而对任意配置预测局部最优词汇量 (适用于过训练/欠训练场景)。

4.2 Over-Tokenized Transformer: Vocabulary is Generally Worth Scaling

动机: Scaling Law 表明更大模型需要更大词汇量, 但未区分输入和输出词汇的不同作用。作者发现: 扩大输入词汇几乎不增加计算成本, 而扩大输出词汇会显著增加训练开销。输入和输出词汇对模型性能的影响规律不同: 大输入词汇始终有益, 大输出词汇在小模型上有害。因此, 需要解耦输入和输出词汇, 分别研究其缩放规律, 并利用大输入词汇提升模型效率。方法: (1) Over-Encoding: 使用层次化 n-gram 输入词汇 (如 1-gram、2-gram、3-gram 组合), 将输入词汇量扩展到数百万 (如 12.8M)。通过矩阵分解 (低秩切片、模运算映射) 高效实现超大嵌入表, 避免参数爆炸。实验发现: 输入词汇量与训练损失呈对数线性关系, 扩大输入词汇可让 400M 模型达到 1B 基线模型的性能。 (2) Over-Decoding: 利用 n-gram 输出词汇提供更细粒度的监督信号, 通过乘积分解将 V^n 大小的输出嵌入表分解为 n 个 V 大小的嵌入表, 将 n-gram 预测损失分解

为多个 next-token 预测损失的组合，仅对足够大的模型有效（小模型上反而有害）。(3) Over-Tokenized Transformer：结合 OE 和 OD（采用 DeepSeek-V3 中的 MTP-DS 形式），进一步提升模型性能。

4.3 Minimax: Visual Tokenizer 的 Scaling Law

为了实现有效的图像生成，潜在空间必须简洁地表征高级语义，实现了图文对比、自我监督和重建损失的联合优化。在 DiT 训练中，相同的 Flops 时，VTP scaling 有助于更好的生成。传统的 Autoregressive 无法扩展用于扩散生成式模型。

5 新闻热点：Minimax 报告——为什么大模型不认识马嘉祺？

5.1 现象

模型能够讲清马嘉祺的履历，但是打不出来“嘉祺”这两个字。同时日语 prompt 会蹦俄文，混效率高达 47%。

5.2 Minimax Debug

「嘉祺」被分词为一个独立的 token (id=190467)。这两个字在日常语料中共现频率较低，作为单一 token 存在略显意外。由此产生一个假设：是否是因为预训练和后训练+serving 阶段使用了不同的 tokenizer 策略，在预训练语料中「嘉祺」实际被切分为两个 token [『嘉』, 『祺』]，导致“嘉祺”这个 token 没有得到充分训练。而后训练和 serving 阶段使用合并后的 token，其生成概率会很低（小于 5%），那么在 top-p = 0.95 的采样策略下就会被 mask，从而无法生成。统计 pretrain 的 emb tokens 和 lm_head Norm 发现这个词的向量范数落在正常范围，并且这个 token 的 top 10 token 是“亚轩、千禧、祺、耀文、嘉”等，说明预训练模型已经能够建立合理的语义分类。定位到是后训练 SFT 时 lm_head 发生了偏移，因为 SFT 数据中“嘉祺”的 token 覆盖远低于预训练（后训练的包含“嘉祺”的样本不足 5 条），导致低频中文 token 训练在向量空间发生了挤压，使得 lm_head 对生僻 token 几乎没有区分度。

5.3 解决方案

方案一：分割全量词表，全词表随机划分为若干份，每份大约有 8k 个 token，每一份 token 都加上“请重复以上内容”的 query，监督输出为相同的划分词表。通过简单的复读任务，保证了 SFT 阶段每个 token 生成频率的下限。方案二：删除构建词表阶段的尾分布，即那些在后训练中没有场景的词。本质上这个问题就是 BPE 和训练的不匹配导致的（Claude Opus 4.6 到 4.7 的更新也删除了部分词表，可能也是考虑到了“马嘉祺”的问题）。最后使用 Option1 的实验效果：输出空间中相似度小于 0.95 的 token embedding 数量减少，日俄语输出混淆比例下降，模型能够说出“马嘉祺”三个字了。

5.4 联想到一篇论文：Lost in Backpropagation: The LM Head is a Gradient Bottleneck

论文的核心论点是：在语言模型的最后一层（LM Head，即从隐藏维度 D 映射到词表大小 V 的线性层），由于 D 远小于 V ，反向传播的梯度在通过这个低秩层时会受到严重压缩，导致 95-99% 的梯度信息丢失。丢失的部分主要对应于词表中那些“尾部”的、不常见的 token。

论文现象：在“SpamLang”实验中，当词表 V 相对于隐藏维度 D 过大时（例如 $V = 131072, D = 576$ ），模型甚至连“重复同一个随机 token”这种极其简单的模式都难以学会，表现为训练缓慢、对某些 token 的生成能力极差（这个现象与马嘉祺现象是统一的）。

论文解释：对于像“嘉祺”这样的低频 token，其对应的 logit 梯度本身就是微弱且

稀疏的（因为它在训练数据中出现次数少）。当这个微弱的梯度信号通过 LM 头这个低秩“瓶颈”时，它大部分能量会被投影到 W 矩阵的零空间中，被彻底丢弃，留下的只是少量噪声。因此，模型很难为这个 token 建立稳定、有区分度的嵌入。论文提出的解决方案是改进 LM 头设计（如更高秩的投影）或优化梯度传递。

6 对推荐系统的启发

6.1 ActionPiece——BPE 分词的 GR

动机：现有方法中相同的物品在不同用户序列中总是被 token 化为相同的 token。这种方式忽略了行为所处的上下文信息，例如：同一个 T 恤在“运动装备序列”和“时尚搭配序列”中可能有完全不同的语义意义。这种上下文无关的 tokenization 方式限制了模型对用户意图的理解能力。早期 NLP 使用上下文无关的 tokenization（如 word-level），后来发展为上下文感知的 BPE、Unigram 等。论文提出：推荐系统中的行为序列也应当具备上下文感知的 tokenization 能力。

方法：(1) 行为表示为无序特征集合：每个物品表示为多个特征（如品牌、颜色、价格等）的无序集合，用户行为序列变为 $S' = \{A_1, A_2, \dots, A_t\}$ ，其中每个 A_i 是一个特征集合。

(2) 上下文感知的词汇构建：初始化每个特征作为一个初始 token；迭代合并时统计 token pair 的共现频率，但有两种类型：同一集合内的 token 对，相邻集合间的 token 对；对不同类型 token 对赋予不同权重（基于集合大小的概率加权）；合并频率最高的 token 对，生成新 token；引入中间节点：当合并来自两个不同行为的 token 时，插入一个中间节点，表示跨行为的特征组合。

(3) 集合排列正则化 (SPR)：由于每个行为的特征是无序的，ActionPiece 会随机排列每个集合内的特征顺序，然后将排列后的集合序列展平为一维 token 序列，使用 BPE 方法对该序列进行 segmentation。同一个行为序列可以生成多个语义相同但 token 顺序不同的序列，作为数据增强。

(4) 生成式推荐模型的训练与推理：训练时对每个行为序列生成多个 SPR 版本的 token 序列，使用 Transformer 进行自回归生成；推理时对同一输入生成多个 SPR 版本，分别推理后 ensemble，提升推荐性能。